

I-Tests de syntaxe

I-1-Tests valides (syntax et lex)

progvide.deca : programme vide
hello.deca : chaîne hello world
id_1.deca : test entier
float_1.deca : test flottant
float_2.deca : test flottant avec suffixe f autorisé
float_3.deca : test flottant avec notation scientifique
float_4.deca : test flottant (hexadecimal)
char_1.deca : test chaînes de car valides
world_import.deca : test pour #include

inst_after_decl.deca : Declaration puis utilisation

I-2-Tests invalides (syntax et/ou lex)

SYNTAXE

erreur_id_4.deca : format identificateur invalide
erreur_if.deca : erreur syntaxe : pas de parenthèses autour de la condition du if
erreur_mot_res_1.deca : erreur syntaxique, mot réservé en tant qu'identificateur
erreur_mot_res_k.deca avec k entre 1 et 20 : Idem
erreur_ptvirg_1.deca : ok lex, erreur syntaxe : point virgule manquant après la première instruction
erreur_ptvirg_2.deca : ok lex, erreur syntaxe: point virgule au mauvais endroit
erreur_while.deca : ok lex, erreur syntaxe: déclaration après while sans bloc d'accolades

missing_rbrace.deca : accolade manquante
missing_virgule.deca : virgule manquante
missing_left_operand.deca : la valeur de gauche manque après le =
empty_assign.deca : variable non déclarée
block_imbr_mixtype.deca : bloc imbriqué
block_imbr_prob.deca : bloc imbriqué
decl_after_instr1.deca : println puis affectation
decl_after_instr2.deca : variable non déclarée
double_operator_mult : opérateur successifs
double_operator_plus : opérateur successifs
invalid_lvalue : affectation à un nombre
print_no_parenthesis : print + pas de parenthèses
println_no_parenthesis: println + pas de parenthèses

LEXICAL

simple_var_1.deca : erreur lex, entier commençant par 0
erreur_float_1.deca : erreur lex, flottant trop grand
erreur_float_2.deca : erreur lex, flottant invalide
erreur_id_2.deca : erreur lex, format identificateur invalide
erreur_id_3.deca : erreur lex, format identificateur invalide
invalid_char.deca : erreur lex, caractère invalide ù
invalid_id_digit.deca : identificateur incorrect

chaîne_incomplete : erreur lex, chaîne non terminée
simple_lex.deca : ok lex; erreur syntaxe

simple_exp_1.deca : ok lex, erreur syntaxe
erreur_id_r lex, format identificateur invalid
erreur_char_1.deca : erreur lex, non respect de la convention \\ ou \”
erreur_char_2.deca : erreur lex, guillemet non échappé

II-Tests contextuels

Chemin: [src/test/deca/context](#)

Les tests sans objet sont basés sur la “liste des erreurs pour la partie B (sans objet)” fournie durant le suivi 1.

II-1-Tests valides

Chemin: [src/test/deca/context/valid](#)

II-1-1-Tests spécifiques pour la partie sans objet (dossier sans-objet)

Chemin: [src/test/deca/context/valid/sans-objet](#)

format utilisé ici est “NombrePasse-NombreRègle-Description.deca : Description du test”

0-1-id-declare.deca : test règle 0.1 utilisation d’une variable bien déclarée
0-2-type-connu.deca : test règle 0.2 utilisation type de base (int float et booléen)
3-8-init-ok.deca : initialisation d’un float par un int
3-17-declare-type-nonvoid.deca : test règle 3.17 déclaration variable type non void
declarations-uniques.deca : déclarations d’identificateurs différents dans un bloc
3-28-affect-ok.deca : affectation compatible (int vers float)
3-29-if-boolean.deca : condition booléenne correcte dans un if
3-29-while-boolean.deca : condition booléenne correcte dans un while
3-31-print-ok.deca : print sur types autorisés (int float et string)
3-31-printlnx-num.deca : printlnx sur types numériques (int et float)

3-32-lvalue-valide.deca : affectation sur une variable (lvalue valide)
3-33-add-compatible.deca : addition de types compatibles (int et float)
3-33-and-boolean.deca : opérateur && sur booléens
3-33-or-boolean.deca : opérateur || sur booléens
3-33-modulo-int.deca : opérateur modulo sur des entiers
3-37-unary-minus-num.deca : opérateur unaire '-' sur int et float
3-37-not-boolean.deca : opérateur '!' sur booléen
3-39-cast-valide.deca : cast valide (float vers int)

II-1-2-Tous les autres tests

Chemin: [src/test/deca/context/valid](#)

ConcatenationCompatible1.deca : Concaténation Compatible de chaîne de caractère
ConcatenationCompatible2.deca : Concaténation Compatible de chaîne de caractère
float_init_with_int.deca : Conversion automatique du type
if_mixed_comparison.deca : Condition if et comparaison
int_to_float_assign.deca : affectation Compatible
int_to_int_assign.deca : affectation Compatible
mixed_arithmetic_expr1.deca : opération et type
mixed_arithmetic_expr2.deca : float + int = float.

Chemin: [src/test/deca/context/valid/mine](#)

Communs aux trois passes

Passe 1

1-1valid.deca : règle 1.1 avec programme vide donc valide
1-2valid.deca : règle 1-2 déclaration classes enchaînées

Passe 2

...

Passe 3

3-1valid.deca : simple test règle 3.1
3-2valid.deca : simple test règle 3.2
3-3valid.deca : test règle 3.3 avec programme vide
3-5valid.deca : simple test règle 3.5
3-6valid.deca : simple test règle 3.6
3-8valid.deca : test règle 3.8 avec un sous-type
3-9valid.deca : test règle 3.9 avec champ sans initialisation
3-14valid.deca : simple test règle 3.14
3-15valid.deca : test règle 3.15 méthode avec corps Assembleur
3-17valid.deca : simple test règle 3.17
3-18valid.deca : test règle 3.18 avec blocs imbriqués

3-22valid.deca : test règle 3.22 (visibilité dans les blocs if/else)

3-23valid.deca : test règle 3.23 avec instruction vide

II-2-Tests invalides

Chemin: [src/test/deca/context/invalid](#)

II-2-1-Tests spécifiques pour la partie sans objet (dossier sans-objet)

Chemin: [src/test/deca/context/invalid/sans-objet](#)

format utilisé ici est "NombrePasse-NombreRègle-Description.deca : Description de l'erreur"

0-1-id-non-declare.deca : règle 0.1 utilisation variable non déclarée

0-2-type-inconnu.deca : règle 0.2 utilisation type inconnu

3-8-init-incompatible.deca : initialisation incompatible (variable type 'int' avec expr type 'string')

3-17-variable-void.deca : règle 3.17 déclaration variable type void

3-17-double-declaration.deca : double déclaration dans même bloc

3-24-return-missing.deca : return manquant dans méthode non-void

3-28-affectation-incompatible.deca : affectation de types incompatibles

3-28-return-mismatch.deca : type de retour incompatible (float vers int)

3-32-lvalue-invalide.deca : lvalue invalide (affectation sur littéral)

3-29-if-non-boolean.deca : condition non booléenne dans if

3-29-while-non-boolean.deca : condition non booléenne dans while

3-31-print-boolean.deca : print sur type non imprimable

3-31-println-string.deca : println sur type non numérique

3-33-add-incompatible.deca : addition de types incompatibles

3-33-modulo-float.deca : opérateur modulo sur flottants

3-37-unary-minus-boolean.deca : opérateur unaire '-' sur booléen

3-37-not-non-boolean.deca : opérateur '!' sur non-booléen

3-33-and-non-boolean.deca : opérateur && sur non-booléens

3-33-or-non-boolean.deca : opérateur || sur non-booléens

3-39-cast-impossible.deca : cast impossible (string vers int)

3-40-instanceof-non-classe.deca : instanceof avec types non-classes

3-43-this-main.deca : utilisation de 'this' en programme principal

Chemin: [src/test/deca/context/invalid](#)

and_invalid_type.deca : Condition if et &&

doubleDeclaration.deca : Double declaration

float_to_int_assign.deca : probleme de typage;

if_condition_int.deca : Condition if et type different de bool

int_string_init.deca : incompatibilité type

int_to_bool.deca : affectation à un boolean

modulo_float.deca : modulo sur un float

noDeclaration.deca : variable non déclarée

op_invalid_type.deca : incompatibilité type

println_string_plus_int.deca : incompatibilité type
void_assignment.deca : incompatibilité type

II-2-2-Tous les autres tests

Chemin: [src/test/deca/context/invalid/mine](#)

Communs aux trois passes

Passe 1

1-1invalid.deca : règle 1.1 avec déclaration classe après bloc MAIN
1-3invalid.deca : règle 1-3 avec héritage d'un type qui n'est pas une classe
1-3invalidbis.deca : règle 1-3 avec héritage d'une classe non définie
1-3invalidbisbis.deca : règle 1-3 déclaration classes avec un doublon

Passe 2

...

Passe 3

3-1invalid.deca : règle 3.1 sans bloc main
3-1invalidbis.deca : règle 3.1 avec mauvais ordre des blocs
3-4invalid.deca : règle 3.4 avec un retour dans le main (ne respecte pas type void)
3-11invalid.deca : règle 3.11 avec un type de retour inexistant
3-12invalid.deca : règle 3.12 avec un doublon de paramètres
3-15invalid.deca : règle 3.15 méthode avec corps Assembleur (autre type que StringLiteral après asm)
3-21invalid.deca : Test echec règle 3.21: impression d'un objet
3-24invalid.deca : Test blocs et instructions, règle 3.24, return avec valeur dans une méthode void.
3-24invalidbis.deca : Test règle 3.24 avec return interdit dans le main
3-25invalidbis.deca : Test echec règle 3.25, portée des variables définies dans un While